Task 4 – Test-Driven LLM Code Generation

COMP5111 Assignment 2

ZIMO JI, HKUST, Hong Kong SAR

The companion `Task4_Iteration_Trace.pdf` *has the verbatim prompt + response + extracted code for every round. This report is the analysis only.*

1 Setup

1.1 Selected methods

I picked ten public-static methods from `Subject.java` and put them in a reduced CUT (`reports/Task4_Data/NewSubject.java`), plus one private helper (`isOneOf`) needed by `parseToken`. All public signatures are byte-identical to the originals:

#	Signature	From `Subject.java`
1	`boolean startsWithIgnoreCase(String, String)`	`StringAlgorithms`
2	`String parseToken(String, char[])`	`StringAlgorithms`
3	`int extractIntInStr(String)`	`StringAlgorithms`
4	`int[] getVersionNo(String)`	`StringAlgorithms`
5	`String padLeft(String, short, char)`	`StringAlgorithms`
6	`boolean judgeLeapYear(int)`	`DateTimeAlgorithms`
7	`int calcDaysInMonth(int, int)`	`DateTimeAlgorithms`
8	`int getQuarter(int)`	`DateTimeAlgorithms`
9	`int monAbbr2month(String)`	`DateTimeAlgorithms`
10	`String month2MonAbbr(int)`	`DateTimeAlgorithms`

I used the Task 2 fault-fixed implementations as ground truth so that EvoSuite's tests pin down the *intended* contract, not the seeded faults.

1.2 EvoSuite generation

```
java -jar lib/evosuite-1.2.0.jar \
   -class comp5111.assignment.cut.NewSubject \
   -projectCP reports/Task4_Data/build/classes \
   -Dsearch_budget=600 -Dstopping_condition=MaxTime \
   -Dcriterion=LINE:BRANCH -Dassertion_strategy=ALL \
   -Drandom_seed=2026
```

100 % LINE coverage (115/115) and 100 % BRANCH coverage (131/131) in 1 s of search; 62 JUnit tests in the 414-line `NewSubject_ESTest.java`.

1.3 LLM endpoint

OpenAI-compatible `/chat/completions`, `temperature=0.1`. Primary model is `gpt-5.1`; I also ran `gpt-5.4` (stronger, footnoted in §3) and `gpt-4o-mini` (weaker, ablation in §6). Raw HTTP from the Python standard library, no SDK.

Author's Contact Information: Zimo Ji, HKUST, Hong Kong, Hong Kong SAR.

2 Prompt design

The same template every round. Four fixed parts; rounds ≥ 2 add two iteration-specific parts:

```
<system>
You are a senior Java engineer. Respond with exactly one fenced ```java code
block as instructed.
</system>

<user>
You are a Java engineer. Implement the class
`comp5111.assignment.cut.NewSubject` so that every JUnit test in the
provided test suite passes.

Rules:
 - Output exactly ONE fenced ```java code block containing the FULL contents
   of NewSubject.java (package, imports if any, the entire class).
 - Do NOT modify the test suite. Do NOT add main(). Keep the package and
   class name unchanged. Method signatures shown below are mandatory.
 - Output nothing outside the single code block.

=== Method signatures + Javadoc (mandatory shape) ===
<NewSubject.java with method bodies stripped to ";">

=== EvoSuite-generated test suite (passes against the ground truth) ===
<entire NewSubject_ESTest.java verbatim>

=== Your previous implementation (round N-1) === # rounds >= 2
<last LLM output>

=== Test failures observed for that implementation === # rounds >= 2
<list of "FAILURE>>>...<<<FAILURE" lines>
</user>
```

I kept the Javadoc on purpose: it carries intent (e.g. "`extractIntInStr` returns the LAST contiguous digit run") that the test suite samples but does not state. Round-1 prompt is 15 138 chars; round 2 grows to about 24 000 once I append the previous attempt and failing tests.

3 First-round result

Metric	Round 1
Compilation OK	yes
Tests run	62
Tests failed	**1**
Pass rate	**98.4 % (61/62)**

The single failure was

```
test51 :: arrays first differed at element [0];
       expected:<6> but was:<316>
```

`test51` calls `getVersionNo("$S3P&,*V+D&1{%U6")` and expects `[6,0,0,0]`. Input has no `.`, so `String.split("\\.", -1)` returns one segment. `gpt-5.1` walked the segment forward and concatenated *every* digit (3, 1, 6 → 316) instead of returning only the last contiguous digit run as the Javadoc says (`". . . &1{%U6"` → 6). The other 61 tests don't exercise this case (they have a leading digit, so forward and backward scans agree, or no digits at all).

Footnote (stronger backend). With `gpt-5.4` and the same prompt, all 62 tests pass on round 1, no iteration needed. Artefacts under `reports/Task4_Data/trial_gpt54/`. I do not use it as the primary experiment because it leaves the iteration loop with nothing to do.

4 Final pass rate after refinement

I appended round 1's failure (`expected:<6> but was:<316>`) to the prompt. Round 2 fixed it: `gpt-5.1` rewrote the digit loop in `getVersionNo` to scan **backwards** from the end of the segment and record the last contiguous digit run. The diff inside `getVersionNo`:

```java
// round 1 (buggy): forward accumulator
for (int j = 0; j < p.length(); j++) {
    char c = p.charAt(j);
    if (Character.isDigit(c)) { hasDigit = true;
        num = num * 10 + (c - '0');
    }
}

// round 2 (fixed): backward search for the last digit run
int end = -1, start = -1;
for (int j = p.length() - 1; j >= 0; j--) {
    char c = p.charAt(j);
    if (Character.isDigit(c)) {
        if (end == -1) end = j;
        start = j;
    } else if (end != -1) {
        break;
    }
}
int num = 0;
for (int j = start; j <= end; j++) num = num * 10 + (p.charAt(j) - '0');
```

Round 2 passes all 62 tests; my loop early-exits.

Final: 100 % (62/62) after 2 rounds.

Per-round artefacts:

Path	Contents
reports/Task4_Data/prompts/round_{1,2}.txt	exact prompts
reports/Task4_Data/llm_outputs/round_{1,2}.txt	raw LLM responses
reports/Task4_Data/generated/round_{1,2}/NewSubject.java	extracted Java each round
reports/Task4_Data/test_results/round_{1,2}.txt	compile + JUnit summary (json)
reports/Task4_Data/final/NewSubject.java	round-2 implementation (= final)
reports/Task4_Data/summary.json	per-round metrics
reports/LLM/Task4_Iteration_Trace.{md,pdf}	full trace

5 Similarities and differences vs Subject.java

I diffed the **final (round 2)** `NewSubject.java` from `gpt-5.1` against the matching ten methods of `Subject.java`.

5.1 Identical (modulo whitespace / comments)

`judgeLeapYear`, `calcDaysInMonth`, `getQuarter`, `month2MonAbbr`, `padLeft`. Same control flow, same constants.

5.2 Same intent, different idiom

- `startsWithIgnoreCase`. Original copies `prefix.length()` chars and compares; `gpt-5.1` walks the prefix with `Character.toLowerCase(c1) != Character.toLowerCase(c2)`. The LLM's loop avoids the substring allocation, behaviour is the same.
- `getVersionNo`. Original walks with `indexOf('.')` and a manual cursor in a `do . . . while`; `gpt-5.1` (round 2) uses `versionString.split("\\.", -1)`, validates `parts.length` and empty parts, then runs the backward digit-run scan above. Same `int[4]` output.
- `parseToken`. Original returns `str.substring(0, pos)` after a `while` loop; `gpt-5.1` adds null/empty checks and returns `str.substring(0, i)` directly inside the matched-terminator branch. The buggy `pos++` of `Subject.java` line 205 is absent (EvoSuite tested the patched ground truth).
- `extractIntInStr`. Original is a single forward pass that resets the accumulator on every non-digit. `gpt-5.1` does a backward scan for the last digit run, then a small forward accumulator inside it. Same contract, more verbose.

5.3 Different control structure

`monAbbr2month`. Original uses a perfect hash `(ch0«16) | (ch1«8) | ch2` and dispatches on twelve hard-coded integers – one of which (`5465466` for "Sep") is the seeded fault from Task 2. `gpt-5.1` drops the hash and uses a Java 7+ `switch (abrr)`:

```
switch (abrr) { case "Jan": return 1; ... case "Sep": return 9; ... }
```

Same return values, but the perfect-hash trick is gone. The LLM couldn't have known the original used it – EvoSuite tests only observe return values.

5.4 LLM-introduced defensive checks

`gpt-5.1` consistently adds null/empty pre-checks the original sometimes omits (e.g. `parseToken` against null `str` or zero-length `terminators`). Doesn't break tests here, but would change the contract if the spec required NPE on null. The same instinct goes wrong on a weaker model (§6).

6 Ablation: weaker backend model

To get iteration data and check how brittle round-1 convergence is, I re-ran the same prompt template, EvoSuite test suite, and `NewSubject.java` with `gpt-4o-mini`. Everything else (`temperature=0.1`, `max-rounds=5`, OpenAI-compatible endpoint) is identical.

Round	compile_ok	run	fail	pass rate
1	yes	62	2	96.8 %
2	yes	62	2	96.8 %
3	yes	62	2	96.8 %
4	yes	62	2	96.8 %
5	yes	62	2	96.8 %

The weaker model converges fast to a near-correct implementation but **doesn't fix the last two failures** even after four rounds of feedback:

- `test43`: `calcDaysInMonth(-3324, 2)` expected 29, got 28.
- `test51`: `getVersionNo("$S3P&,*V+D&1{%U6")` expected `[6,0,0,0]`, got `null` (`assertNotNull` failure).

Both come from over-defensive code that the model wrote on round 1 and kept every subsequent round:

- In `judgeLeapYear`, `gpt-4o-mini` added an unjustified guard `if (year < 0) return false;`. The Javadoc says nothing about the sign; the Gregorian rule works fine for negative years. With this guard, every negative leap year is mis-classified, so February of −3324 returns 28.
- In `getVersionNo`, `gpt-4o-mini` called `Integer.parseInt(parts[i])` directly instead of the spec's "last digit run" extraction (the original calls `extractIntInStr`). `"$S3P&,*V+D&1{%U6"` has no `.` and no leading digit, so `parseInt` throws `NumberFormatException`, and the LLM's `catch (NumberFormatException e) { return null; }` returns `null` instead of `[6,0,0,0]`.

The model received the failure messages verbatim every round. Round 4 sent it:

```
2 of 62 tests failed.
 - test43(...) :: expected:<29> but was:<28>
 - test51(...) :: null
```

…and `gpt-4o-mini` still produced "fixes" with the same two bugs. The trace shows it rewriting the surrounding code each round (different idioms, slightly different control flow) but leaving the two over-defensive guards untouched. Two takeaways:

(1) **Localising a bug from a high-level test message is hard for weak models.** "expected 29, was 28" doesn't say which line. Working out that the bug is in `judgeLeapYear` (not `calcDaysInMonth`) takes counterfactual reasoning that `gpt-4o-mini` doesn't do reliably.

(2) **The model has a strong prior toward defensive code** that the spec doesn't require, and feedback that the defence is wrong is not enough to override that prior.

`gpt-5.4` (§3 footnote) writes none of these guards and passes round 1 outright; primary `gpt-5.1` needs one round of feedback. Same prompt, same Javadoc, same tests; the difference is purely model capability. Per-round traces are in `reports/LLM/Task4_Iteration_Trace_weak_gpt4o_mini.{md,pdf}` and `reports/Task4_Data/weak_gpt4o_mini/`.

7 Insights

Strengths.

- *Synthesis from spec + tests is fast and accurate.* On a ≈250 LOC CUT with Javadoc + 100 % coverage tests, `gpt-5.1` got 9 of 10 methods right on round 1 and the last one after one round of feedback (§4). `gpt-5.4` passes round 1. Almost no prompt engineering beyond format constraints.
- *Iteration can localise and fix bugs from a high-level test message.* In round 2, `gpt-5.1` took `expected:<6> but was:<316>` and correctly diagnosed the bug in `getVersionNo`'s digit extraction (not the test, not `extractIntInStr`, not `String.split`). Non-trivial inference.
- *Idiomatic API choice.* Modern Java (`switch (String)`, `String.split`, `Character.toLowerCase` loops) where legacy code would not.

Limitations.

- *Hidden invariants are erased.* The perfect-hash dispatch in `monAbbr2month` is gone. A caller that depended on the exact integer constants would break silently. Tests don't observe internal control flow, so test-driven generation can't recover it.
- *No "minimum-change" notion.* Asked to reimplement, the LLM rewrites. For single-line patching (Task 2 elsewhere), this is the wrong tool.
- *Spec-test conflict goes to the test.* If the Javadoc and the EvoSuite tests disagree, the LLM follows the tests and ignores the doc.
- *Iteration doesn't always fix things (§6).* On `gpt-4o-mini` the same two bugs survived four rounds of failure feedback.

Limitations of EvoSuite-generated test cases.

- *Coverage saturates fast on simple methods.* 100 % LINE+BRANCH in one second, but the tests are mostly value-instance assertions (`assertEquals("Sep", month2MonAbbr(9))`) that don't express the contract.
- *No equivalence-class abstraction.* 12 separate tests for `month2MonAbbr` (one per valid month) plus several invalid; a hand-written test would be parametric. The bloat dominates the prompt budget (the test suite is over 95 % of round-1 prompt size).
- *Tests reflect whatever the implementation does.* If the ground truth had been buggy, EvoSuite would have locked the bug into the tests, and the LLM would have reproduced it.

Suggestions for improving success rate.

(1) Always include the Javadoc, not just the tests. This was the biggest lever in my run; without it the perfect-hash dispatch in `monAbbr2month` could easily be reproduced as a buggy table.

(2) In iteration rounds, send only the failing-test bodies + a diff of the previous attempt, not the whole suite. My 11 KB suite is already heavy; on bigger CUTs it would exceed the model's effective context.

(3) For single-line patches, use a different prompt shape – full method, suspected line(s) highlighted, failing test, and an explicit "do not rewrite any other line".
(4) Augment EvoSuite with property-based tests (jqwik etc.). Properties like `monAbbr2month(month2MonAbbr(m)) == m` for $1 \leq m \leq 12$ capture invariants that examples can't.
(5) Add a "minimal-diff" objective in the system message when refactoring or patching.

(Body word count, excluding code: about 1100.)